

Especificação da Linguagem Java Suportada pelo Compilador (Java para C)

ESCOPO E SINTAXE

1. Escopo do subconjunto

Este documento define o comportamento esperado do subconjunto de Java aceito como entrada pelo compilador, cujo objetivo é traduzir código-fonte Java para C. Como C não possui orientação a objetos, o subconjunto é procedural, mapeando métodos estáticos Java para funções C.

Fora de escopo (não suportado)

- Classes definidas pelo usuário, herança, interfaces, polimorfismo
- Generics
- Tratamento de exceções (try/catch/throw)
- Coleções (List, Map, etc.), Streams, Lambdas
- Arrays multidimensionais e ponteiros (arrays simples de 1 dimensão podem ser avaliados como extensão futura)
- Modificadores de acesso além de public/static

Em escopo: tipos primitivos, variáveis, operadores, expressões, estruturas condicionais, estruturas de repetição, métodos estáticos, escopo de bloco, tipagem estática.

2. Tipos

O compilador mapeia diretamente os tipos primitivos de Java para os equivalentes em C.

Tipo Java	Tipo C gerado	Observação
int	int	32 bits
float	float	32 bits
double	double	64 bits
char	char	Java usa 16 bits (UTF-16); a geração para C assume mapeamento 1:1 restrito a ASCII
boolean	int (convenção 0/1)	C sem C99 não tem tipo boolean nativo; o compilador gera int com a convenção 0 (false) e 1 (true)
void	void	Apenas como tipo de retorno de método

Sintaxe (declaração de tipo)

```
<tipo> ::= "int" | "float" | "double" | "char" | "boolean" | "void"
```

Semântica

- Todo tipo é primitivo (value type), sem boxing/unboxing automático (boxing é converter automaticamente um primitivo pra seu wrapper, ex.: int para Integer). Portanto, o subconjunto aceito como entrada não permite Integer, Double, Character, etc., apenas os tipos primitivos.
- Não há inferência de tipo (sem var).

Exemplo

```
int idade;  
double salario;  
char letra;  
boolean ativo;
```

3. Variáveis

Sintaxe

`<declaracao_variavel> ::= <tipo> <identificador> ["=" <expressao>] ";"`

Semântica

- Toda variável precisa ser declarada com um tipo explícito antes do uso (tipagem estática).
- Variáveis locais não têm valor padrão em Java. Diferente de C, onde a variável não inicializada contém lixo de memória mas compila, em Java isso é erro de compilação (definite assignment). O compilador exige que toda entrada Java já tenha as variáveis inicializadas antes do uso, então o C gerado nunca herda esse problema.
- Uma variável só é visível dentro do bloco { } em que foi declarada (ver seção 9).

Exemplo

```
int contador = 0;
double media;
media = 7.5;
```

Campos static declarados em Java fora de métodos, o equivalente mais próximo de uma variável global, são suportados e mapeados para variáveis globais em C (ver seção 3.1).

3.1 Variáveis globais

Java não possui variáveis globais no sentido de C: todo código precisa estar dentro de uma classe. O equivalente mais próximo é um campo static, compartilhado por todos os métodos static da mesma classe. O compilador SUPORTA a tradução de campos static Java para variáveis globais em C, já que C aceita declarações no nível do arquivo, fora de qualquer função.

Sintaxe (Java, entrada)

`<campo_static> ::= "static" <tipo> <identificador> ["=" <expressao>] ";"`

Semântica

- Todo campo static de uma classe Java aceita como entrada é traduzido para uma variável global em C, declarada no nível do arquivo.
- No subconjunto suportado, todo método é static. Um campo static Java já é acessível por qualquer método static da mesma classe, exatamente como uma variável global em C é acessível por qualquer função do arquivo, então a tradução preserva esse comportamento sem ajustes.
- Se o campo static não for inicializado explicitamente em Java, ele recebe o valor padrão do tipo (0, 0.0, false, caractere nulo); a variável global gerada em C recebe a mesma inicialização padrão.

Exemplo (Java, entrada)

```
public class Main {
    static int contador = 0;

    static void incrementa() {
        contador++;
    }

    public static void main(String[] args) {
        incrementa();
        incrementa();
        System.out.println(contador); // 2
    }
}
```

Exemplo (C, gerado)

```
int contador = 0;

void incrementa() {
    contador++;
}

int main() {
    incrementa();
    incrementa();
    printf("%d\n", contador);
}
```

```
    return 0;
}
```

4. Operadores

Categoria	Operadores suportados	Exemplo
Aritméticos	+ - * / %	a + b, a % b
Relacionais	== != < > <= >=	a >= b
Lógicos	&& !	a > 0 && b > 0
Atribuição	= += -= *= /=	x += 1
Incremento/decremento	++ -- (pré e pós-fixado)	i++, --j

Semântica

- / entre dois int faz divisão inteira (trunca); entre double/float faz divisão real, comportamento idêntico ao de C, portanto a tradução é direta.
- % só é válido para tipos inteiros.
- Operadores lógicos && e || têm curto-circuito (a linguagem não avalia a segunda expressão se o resultado já foi decidido pela primeira), igual em C e Java.
- Não há overload de operadores (redefinir o que +, -, == etc. fazem para tipos customizados), pois não é suportado em Java de qualquer forma.

Exemplo

```
int soma = a + b;
int resto = a % 2;
boolean valido = (a > 0) && (b != 0);
```

5. Expressões

Sintaxe (simplificada)

```
<expressao> ::= <termo> ((<op_bin>) <termo>)*
<termo>      ::= <literal> | <identificador> | <chamada_metodo> | "(" <expressao> ")"
```

Semântica

- Precedência de operadores segue a mesma ordem de C/Java:
Parênteses () > Unário ! > Multiplicativos * / % > Aditivos + - > Relacionais (<, <=, >, >=, ==, !=) > Lógico AND && > Lógico OR || > Atribuição =
- Expressões são avaliadas da esquerda para a direita, respeitando precedência e associatividade.
- Toda expressão tem um tipo estático resultante (conhecido antes mesmo do programa rodar), verificado em tempo de compilação pelo compilador, antes de gerar o C equivalente. Se der tipo incompatível, a compilação falha e nenhum C é gerado.

Exemplo

```
int resultado = (a + b) * 2 - c / 2;
boolean condicao = (x > 0) && (y < 10 || z == 0);
```

6. Estruturas condicionais

Sintaxe

```
<if_stmt> ::= "if" "(" <expressao_booleana> ")" <bloco>
["else" (<if_stmt> | <bloco>)]
```

Semântica

- A condição precisa ser do tipo boolean, regra já exigida pela própria linguagem Java na entrada (diferente de C, que aceita int como condição). Na geração para C, o valor boolean vira int (convenção 0 e 1), e a condição gerada continua funcionando normalmente, já que C aceita int como condição.

- else if é apenas um if aninhado no else.

Exemplo

```
if (idade >= 18) {
    System.out.println("Maior de idade");
} else if (idade >= 12) {
    System.out.println("Adolescente");
} else {
    System.out.println("Crianca");
}
```

7. Estruturas de repetição

Suportadas: while e for. (do-while não entra no subconjunto suportado)

Sintaxe

```
<while_stmt> ::= "while" "(" <expressao_booleana> ")" <bloco>
<for_stmt>   ::= "for" "(" [<init>] "; " [<expressao_booleana>] "; " [<incremento>] ")" <bloco>
```

Semântica

- Condição de parada deve ser boolean, mesma regra do if.
- for é açúcar sintático equivalente a um while com inicialização antes e incremento no fim do corpo. A sintaxe de for em C é equivalente, então a tradução é direta.
- break e continue são suportados dentro de laços.

Exemplo

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}

int i = 0;
while (i < 10) {
    i++;
}
```

8. Métodos (equivalente a funções C)

Como o subconjunto de entrada não usa OOP, todo método static Java é traduzido para uma função C. Método static não é uma escolha de projeto orientado a objetos aqui: é apenas a forma exigida pela sintaxe do Java, já que todo código precisa estar dentro de uma classe. O compilador remove essa casca ao gerar C, já que funções em C não pertencem a nenhuma classe.

Sintaxe

```
<metodo> ::= ["public"] "static" <tipo_retorno> <identificador> "(" [<parametros>] ")" <bloco>
<parametros> ::= <tipo> <identificador> ("," <tipo> <identificador>)*
```

Semântica

- Todo método aceito como entrada deve ser static: não há instância de objeto no subconjunto suportado.
- Passagem de parâmetros é por valor para tipos primitivos, mapeamento direto para passagem por valor em C (sem ponteiros/referências no escopo atual).
- return é obrigatório em métodos com tipo de retorno diferente de void; todos os caminhos de execução devem retornar um valor, regra já garantida na análise da entrada Java.
- Sobrecarga de métodos (overloading, vários métodos com o mesmo nome e parâmetros diferentes) é permitida em Java, mas C não suporta funções com nomes repetidos. Decisão de escopo pendente: se o compilador aceitar overload na entrada, cada sobrecarga precisa virar uma função C com nome único (name mangling, ex.: soma_int_int, soma_double_double); se não aceitar, entrada Java com overload deve ser rejeitada.

Exemplo

```
public static int soma(int a, int b) {
    return a + b;
}
```

```

    }

    public static void imprimir(String msg) {
        System.out.println(msg);
    }
}

```

9. Regras de escopo

- **Escopo de bloco:** uma variável declarada dentro de `{ }` só existe dentro daquele bloco e blocos aninhados dentro dele, comportamento equivalente ao de C, então a tradução preserva a mesma estrutura de blocos.
- **Sombreamento (shadowing):** a própria linguagem Java já proíbe redeclarar uma variável já visível no escopo externo com o mesmo nome dentro de um bloco filho. Como o subconjunto só aceita Java válido como entrada, código com shadowing nunca chega ao compilador (é rejeitado antes, pelo próprio compilador Java). C permite shadowing, mas isso não gera conflito na tradução, já que a entrada nunca vem com esse padrão.
- **Escopo de método:** parâmetros e variáveis locais de um método não são visíveis fora dele, mesma regra em C.
- **Escopo de classe (variáveis globais):** não existe variável global no sentido de C dentro da sintaxe Java. Um campo static, o equivalente mais próximo, é traduzido para uma variável global em C (ver seção 3.1).
- **Regra de acesso static:** em Java, um método static só pode acessar diretamente campos que também sejam static. Como todo método do subconjunto é static, todo campo do subconjunto também é static, condição que garante que a tradução para variável global em C funcione sem ajustes adicionais.

Exemplo (código que a própria linguagem Java já rejeita, portanto nunca chega ao compilador)

```

int x = 10;
{
    int x = 20; // ERRO em Java: variável já definida no escopo
    int y = 20; // OK
}

```

10. Regras básicas de tipagem

- **Tipagem estática e forte:** toda variável e expressão tem tipo definido em tempo de compilação; não há coerção implícita solta como em linguagens dinâmicas.
- **Conversões implícitas (widening)** permitidas, na mesma direção de C/Java:
char para int para long para float para double.

Conversões explícitas (narrowing) exigem cast

```

double d = 3.9;
int i = (int) d; // trunca para 3

```

- **Condições booleanas:** a entrada Java sempre usa boolean em if e while, a própria linguagem já exige isso. Na geração para C, esse boolean vira int (convenção 0 e 1), e a condição continua válida, já que C aceita int como condição.
- **Sem coerção automática entre tipos incompatíveis:** char e int interoperam (por causa do valor numérico do char), mas boolean nunca se mistura com tipos numéricos em Java. Essa regra já é garantida antes mesmo do código chegar ao compilador.

Resumo de cobertura (para referência rápida)

Construção	Suportado
Tipos primitivos (int, double, float, char, boolean, void)	Sim

Variáveis locais	Sim
Operadores aritméticos/relacionais/lógicos/atribuição	Sim
Expressões com precedência	Sim
if / else if / else	Sim
while / for	Sim
do-while	a definir
break / continue	Sim
Métodos estáticos com parâmetros e retorno	Sim
Sobrecarga de métodos (overloading)	não
Escopo de bloco e de método	Sim
Campos static (variáveis globais)	Sim
Classes, herança, interfaces	Não
Arrays	A definir
Exceções	Não
Generics / Collections / Streams / Lambdas	Não